Induction Head Toxicity Mechanistically Explains Repetition Curse in Large Language Models

Shuxun Wang[§*] **Qingyu Yin**[§*] **Chak Tou Leong**[†] **Qiang Zhang**[§] **Linyi Yang**[‡]
[§] Zhejiang University [†] The Hong Kong Polytechnic University
[‡] University College London

arXiv:2505.13514v1 [cs.CL] 17 May 2025

Abstract

Repetition curse is a phenomenon where Large Language Models (LLMs) generate repetitive sequences of tokens or cyclic sequences. While the repetition curse has been widely observed, its underlying mechanisms remain poorly understood. In this work, we investigate the role of induction heads—a specific type of attention head known for their ability to perform in-context learning—in driving this repetitive behavior. Specifically, we focus on the "toxicity" of induction heads, which we define as their tendency to dominate the model's output logits during repetition, effectively excluding other attention heads from contributing to the generation process. Our findings have important implications for the design and training of LLMs. By identifying induction heads as a key driver of the repetition curse, we provide a mechanistic explanation for this phenomenon and suggest potential avenues for mitigation. We also propose a technique with attention head regularization that could be employed to reduce the dominance of induction heads during generation, thereby promoting more diverse and coherent outputs.

1 Introduction

Large Language Models (LLMs) (Vaswani et al., 2017; Yin et al., 2024a,b; Taori et al., 2023; Brown et al., 2020) are designed to generate coherent, controlled, and contextually appropriate responses. However, we identify a critical phenomenon termed the *Repetition Curse*. Repetition curse refers to the phenomenon that LLMs repeat one or several tokens until reaching the context window limitation. Previous work has linked excessive repetition to overly intense model steering (Turner et al., 2023), the use of Sparse Autoencoders (SAEs) with repetitive features (Yao et al., 2025), or underfitting during model training. Our research explores a distinct way this phenomenon can manifest: by deliberately crafting specific input prompts. We have found that certain prompts can reliably trigger models to enter these repetitive loops, and this is strongly associated with in-context learning.

At the heart of this issue lies the transformer's induction head, a specialized attention structure that identifies and replicates subsequences from prior contexts (Olsson et al., 2022). By matching historical token patterns to predict subsequent tokens, induction heads facilitate ICL and repetitive tasks. Yet, repetition becomes detrimental when models prioritize redundant patterns over novel or contextually relevant outputs: the Repetition Curse. While prior work investigates repetition through neuron-level analyses (Hiraoka and Inui, 2024) and self-reinforcement mechanisms (Li et al., 2023), the root cause remains inadequately explained.

We conducted a series of experiments to investigate this phenomenon, employing diverse tokens and prompts. Our findings demonstrate that the phenomenon is easily replicable across LLMs of varying architectures and parameter sizes. Given the close relationship between repetition and induction heads, we propose the **Induction Head Toxicity Theory**, which suggests that the overdominance of induction heads during text generation exacerbates repetitive outputs. Specifically, induction heads suppress contributions from other attention heads, enforcing rigid pattern replication and limiting diversity. To quantify the influence of induction heads during repetitive generation, we calculate their causal influence with activation patching (Heimersheim and Nanda, 2024). Finally, we evaluate mitigation strategies, such as attention head contribution scaling and dynamic head pruning, to recalibrate the influence of induction heads and restore balanced, diverse text generation.

[*]Equal contribution.

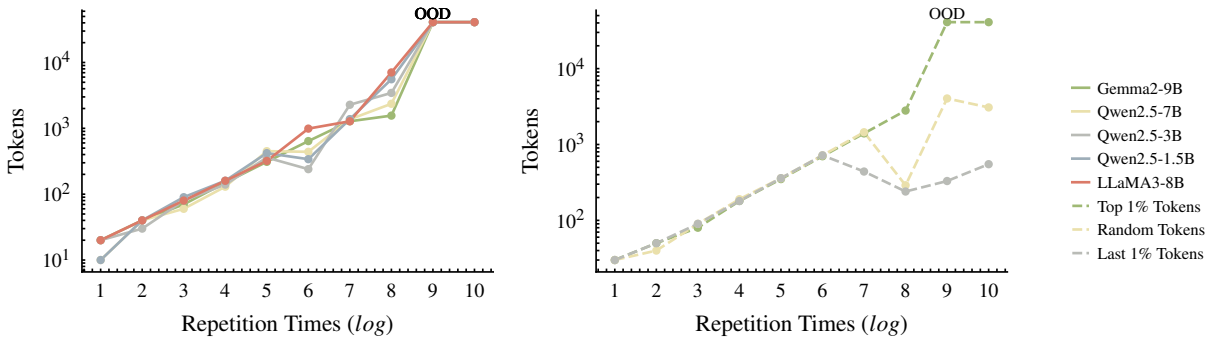

Figure 1: **Left**: Trend of LLM's final response length with increasing repetition requests, demonstrating a sudden explosion in response length when the number of repetitions is continuously increased. **Right**: Average response length for tokens with the highest (top 1%) and lowest (bottom 1%) appearance frequencies in the pretraining corpus. **OOD** indicates the LLM's generation exceeded the set maximum text window. All models mentioned above are instructed versions.

2 Background

We will first examine the repetition curse issue in this Section. In Section 2.1, we provide settings of our generation, models, and datasets that can replicate the repetition curse. In Section 2.2, we analyze the experiments and give some empirical findings of this issue.

2.1 Settings

Generation. We designed a controlled generation experiment to investigate the repetition curse. Our approach focuses on quantifying the models' repetition times when prompted to repeat it a variable number of times. We define *repetition times* as the number of times the target token sequence should be repeated. Our prompt construction process starts with a template that includes clear instructions for the LLM. The target token itself is randomly selected from a predefined vocabulary of tokens (See Figure 6). This vocabulary includes a variety of characters, numbers, common words, and even some special symbols for ensuring a diverse range of inputs for the LLM. The random selection process helps to mitigate any potential bias towards specific tokens. We also choose the last 1% token in Redpajama with n-gram counting (Liu et al., 2024), for testing the corner cases in repetition generation. The number of repetitions is a key parameter in our experiments, systematically varied to explore how it affects the LLM's performance. We select the repetition times from 2 to 512, increasing with a 2-based exponential scale. For better instruction and format following, we prefilled 8 tokens when the repetition count is larger than 8.

Model and Dataset. To evaluate the repetition curse phenomenon across various scales and architectures, we select a diverse set of open-source LLMs for our experiments. Our selection criteria prioritize models with varying parameter counts and architectural designs to provide a comprehensive analysis of the repetition curse. The models included in our study are: (i) Qwen2.5-Instruct (Yang et al., 2024) in 1.5B, 3B, and 7B parameter sizes. (ii) LLaMA3-Instruct (He et al., 2024) with 8 billion parameters. (iii) Gemma2-it (Team et al., 2024) with 9B parameter. In addition to model selection, we curate a dataset of diverse evaluation tasks to assess the general response length. The benchmarks used in our study include MMLU-Pro (Wang et al., 2024), HumanEval+ (Liu et al., 2023), and ARC Challenge (Clark et al., 2018).

2.2 Experimental Results and Analysis.

In a nutshell, we have three key findings:

Generality Across Models and Scale. Our experiments reveal a striking consistency in the repetition curse across different models and scales (illustrated in the left of Figure 1). The models seem unable to stop the repetition once it has begun, even when the target count is surpassed. This phenomenon is observed across all model sizes tested, from the smaller 1.5B parameter Qwen model to the larger 9B parameter Gemma-2. This universality indicates that the repetition curse is not a size-specific artifact, but rather a fundamental challenge for LLMs, irrespective of scale.

Not a Counting Issue. With our prompt requesting the LLM to repeat the token in certain times, it is needed to make sure that the repetition curse

is not a counting issue LLMs fail to estimate how many tokens they have generated. We found that the LLM's struggle is *not* due to *an inability to accurately count or measure* the required number of repetitions. Instead, it points towards a lack of control over the generation process itself. There are two evidences: First, the generated repetition times predominantly exceed the specified repetition times, rather than falling short. This are verified in all models with different architectures and parameter sizes. Second, we observed a "burst" effect: as the target repetition times increase, the extent of over-repetition grows dramatically. Beyond a certain threshold, the models appear to "lose control," generating far more repetitions than requested.

Vocabulary Sensitivity. The LLM's performance in handling the repetition task is not uniform across the token vocabulary (See Figure 1). Common, frequently encountered tokens, especially numbers and simple words, often make LLMs prematurely fallen into the repetition process. However, for less frequent, more obscure tokens, the LLM often prematurely halts the repetition process. It seems to "give up" on repeating these less common tokens. This vocabulary sensitivity further complicates the repetition curse phenomenon and points to potential weaknesses in the LLM's handling of the long tail of the vocabulary distribution.

3 Induction Head Toxicity Theory

In the last section, we empirically analyze the repetition curse. In this Section, we proposed the **Induction Head Toxicity Theory** for demystifying this phenomenon from a perspective of *induction heads*. In Section 3.1, we give a pipeline for detecting the induction heads in transformers with causal intervening patching techniques. In Section 3.2, we formalize the toxicity of induction head. In Section 3.3, we experimentally verify our theory.

3.1 Identifying and Analyzing Induction Heads.

Repetition is a special form of in-context learning (ICL), and the phenomenon of ICL in large language models relies heavily on specialized attention mechanisms called *induction heads*. These components enable models to detect and replicate patterns from context without weight updates. Our investigation employs controlled experiments and activation analysis to characterize these critical components.

Operational Definition of Induction Heads. Induction heads operate through two complementary mechanisms: prefix matching and token copying. The prefix matching refers to a head's ability to attend to the first occurrence of a token in repetitive patterns (*e.g.,* recognizing the initial $[A]$ in $[A]\ [B] \cdots\ [A]$). The copying measures how effectively the head increases the logit probability for the subsequent token (*e.g.,* promoting $[B]$ after the second $[A]$). Formally, for a pattern $[A]\ [B] \cdots [A]$, let p_{match} denote the attention probability assigned to the first $[A]$, and $\Delta\ell_{\text{copy}}$ represent the logit boost for $[B]$. Effective induction heads maximize both p_{match} and $\Delta\ell_{\text{copy}}$ simultaneously [1].

Randomized Copy Task Design. To isolate induction head behavior from semantic influences, we construct a synthetic copy task using randomly selected tokens. Let $\mathbb{V}$ be the model's vocabulary and L the sequence length. Each trial generates a string:

$$x_0, x_1, \ldots, x_n, x_{\text{prefix}}, x_{\text{copy}}, x_{n+1}, \ldots, x_L, x_{\text{prefix}}$$

where $\{x_i\}_{i=0}^{L}$ are distinct tokens randomly sampled from $\mathbb{V} \setminus \{x_{\text{prefix}}, x_{\text{copy}}\}$. This structure creates a non-trivial copying challenge where successful prediction of x_0 after the final x_{prefix} requires recognizing the prefix-copy pattern amidst random noise.

Activation Patching Methodology. We develop a three-stage causal intervention protocol based on activation patching to identify critical components: (i) *Clean Forward Pass.* Process a valid copy sequence S_{clean} through the model, caching activations $A_{\text{clean}}^{(i,j)}$ for each attention head (i, j) and saving the logits ℓ_{clean}. (ii) *Corrupted Forward Pass.* Process a modified sequence S_{corrupt} where the token x_{copy} is replaced with a random token, recording baseline logits ℓ_{corrupt} and activations $A_{\text{corrupt}}^{(i,j)}$ (iii) *Patching Intervention.* For each head (i, j), reprocess S_{corrupt} while replacing $A_{\text{corrupt}}^{(i,j)}$ with $A_{\text{clean}}^{(i,j)}$, obtaining patched logits $\ell_{\text{patched}}^{(i,j)}$.

Definition 3.1 (Induction Causal Importance of Attention Head)**.** The causal importance $I_{(i,j)}$ of

[1] Here we only discusses about the behavioral induction heads, which are slightly different from those mechanistically defined induction heads from a 2-layer pretrained transformers (Olsson et al., 2022). We also do not intend to classify them into two types, as this contributes less to analysis.